# Digital Systems

## Lecture 10

## Programming with Interrupts

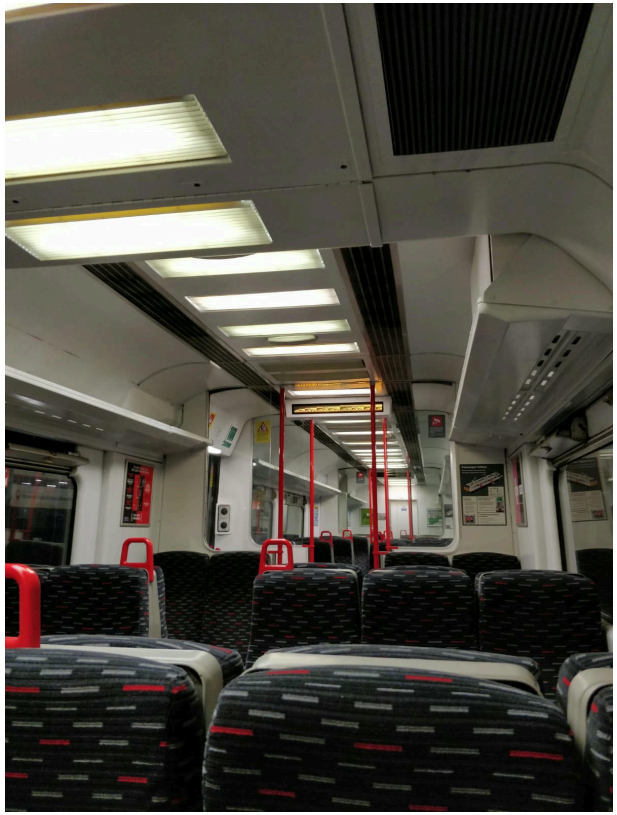
Mark van der Wilk

Hilary Term 2025

Department of
COMPUTER
SCIENCE

Following the course of past years by Mike Spivey

# More Chocolate



What is going on here? Chocolate for the best hypothesis.

**? Do we need to wait around while UART is transmitting?**

What is the point of having hardware to handle receiving/transmission, if the CPU will just wait around until it's done anyway?

**? Do we need to wait around while UART is transmitting?**

What is the point of having hardware to handle receiving/transmission, if the CPU will just wait around until it's done anyway?

```
void serial_putc(char ch) {
    while (! UART_TXDRDY) { /* idle */ } UART_TXDRDY = 0;
    UART_TXD = ch;
}
```

**Do we need to wait around while UART is transmitting?**

What is the point of having hardware to handle receiving/transmission, if the CPU will just wait around until it's done anyway?

```
void serial_putc(char ch) {
    while (! UART_TXDRDY) { /* idle */ } UART_TXDRDY = 0;
    UART_TXD = ch;
}
```

**Hardware should operate at the same time as CPU is executing useful instructions.**

There was a *tiny* bit of parallelism: The last character to be sent!

# Today

**? How can we respond to events (that hardware notices), without making the software do nothing while it waits?**

- 
- 
- 
- 

# Today

**? How can we respond to events (that hardware notices), without making the software do nothing while it waits?**

We will consider various solutions, keeping in mind *design constraints*.

- 
- 

- 
- 

# Today

**? How can we respond to events (that hardware notices), without making the software do nothing while it waits?**

We will consider various solutions, keeping in mind *design constraints*.

We want:

- 
- 
- 
- 

# Today

**? How can we respond to events (that hardware notices), without making the software do nothing while it waits?**

We will consider various solutions, keeping in mind *design constraints*.

We want:

- to avoid making the CPU wait until some event occurs in hardware,
-
-
-

# Today

**? How can we respond to events (that hardware notices), without making the software do nothing while it waits?**

We will consider various solutions, keeping in mind *design constraints*.

We want:

- to avoid making the CPU wait until some event occurs in hardware,
- to be able to deal with many different types of events (not just UART, but e.g. also noticing when a button is pressed),
-
-

# Today

**? How can we respond to events (that hardware notices), without making the software do nothing while it waits?**

We will consider various solutions, keeping in mind *design constraints*.
We want:

- to avoid making the CPU wait until some event occurs in hardware,
- to be able to deal with many different types of events (not just UART, but e.g. also noticing when a button is pressed),
- to have a minimal waiting time to respond to an event,
-

# Today

**? How can we respond to events (that hardware notices), without making the software do nothing while it waits?**

We will consider various solutions, keeping in mind *design constraints*.
We want:

- to avoid making the CPU wait until some event occurs in hardware,
- to be able to deal with many different types of events (not just UART, but e.g. also noticing when a button is pressed),
- to have a minimal waiting time to respond to an event,
- software that is modular, in the sense that it doesn't need to be adapted when other parts of the program change which events they deal with.

# One Possible Solution

- `serial_putc()` no longer waits and sends character.
-
-

# One Possible Solution

- `serial_putc()` no longer waits and sends character.
- Instead, it places a character in a buffer.
-

# One Possible Solution

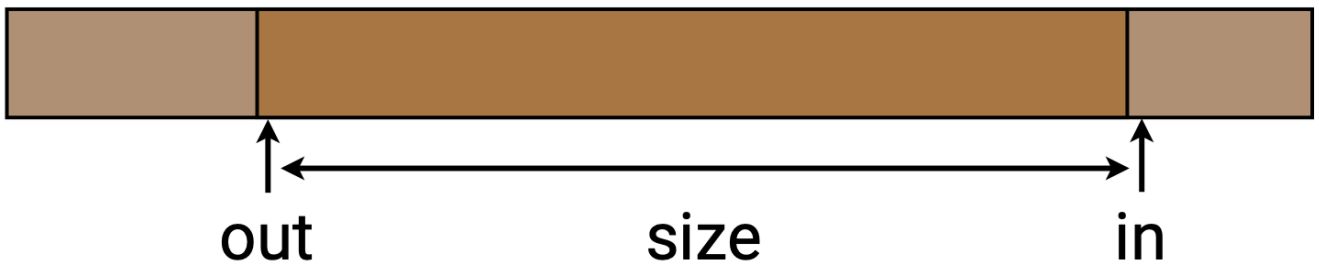
- `serial_putc()` no longer waits and sends character.
- Instead, it places a character in a buffer.
- Throughout the program, check if UART is ready, and send character if so.

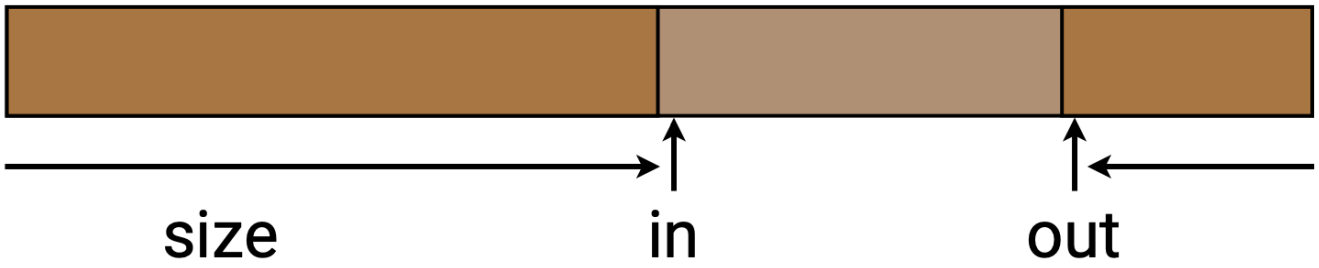
# One Possible Solution

- `serial_putc()` no longer waits and sends character.
- Instead, it places a character in a buffer.
- Throughout the program, check if UART is ready, and send character if so.

```
#define NBUF 64
static int bufcnt = 0;
static int bufin = 0;
static int bufout = 0;
static char txbuf[NBUF];
```

# Circular Buffer




- Add characters at `bufin`.

- Send characters from `bufout`.

# One Possible Solution

```
void serial_putc(char ch) {
  while (bufcnt == NBUF) { pause(); }   // Buffer is full...
  txbuf[bufin] = ch; bufcnt++;
  bufin = (bufin+1) % NBUF;
}
```

```
int prime(int n) {
    int k = 2;

    while (k * k <= n) {
        if (n % k == 0) return 0;
        poll_uart();
        k++;
    }

    return 1;
}
```

```
void poll_uart(void) {
    if (UART_TXDRDY) {
        // send another char
    }
}
```

# What do we think of this solution?

We want:

- 
- 
- 
- 

# What do we think of this solution?

We want:

- ✅ to avoid making the CPU wait until some event occurs in hardware,
-
-
-

# What do we think of this solution?

We want:

- ✅ to avoid making the CPU wait until some event occurs in hardware,
- ❓ to be able to deal with many different types of events (not just UART, but e.g. also noticing when a button is pressed),
-
-

# What do we think of this solution?

We want:

- ✅ to avoid making the CPU wait until some event occurs in hardware,
- ❓ to be able to deal with many different types of events (not just UART, but e.g. also noticing when a button is pressed),
- ❓ to have a minimal waiting time to respond to an event,
-

# What do we think of this solution?

We want:

- ✅ to avoid making the CPU wait until some event occurs in hardware,
- ❓ to be able to deal with many different types of events (not just UART, but e.g. also noticing when a button is pressed),
- ❓ to have a minimal waiting time to respond to an event,
- ❌ software that is modular, in the sense that it doesn't need to be adapted when other parts of the program change which events they deal with.

## Better Solution: Interrupts

Hardware *interrupts* the normal execution of instructions in response to an event, and starts executing a different subroutine.

- 
-

## Better Solution: Interrupts

Hardware *interrupts* the normal execution of instructions in response to an event, and starts executing a different subroutine.

- When `UART_TXDRDY` becomes true, execute `uart_handler()`.
-

## Better Solution: Interrupts

Hardware *interrupts* the normal execution of instructions in response to an event, and starts executing a different subroutine.

- When `UART_TXDRDY` becomes true, execute `uart_handler()`.
- Variables that are shared between interrupt and main code, must be marked `volatile` (can change any time, outside direct code execution).

## Better Solution: Interrupts

Hardware *interrupts* the normal execution of instructions in response to an event, and starts executing a different subroutine.

- When `UART_TXDRDY` becomes true, execute `uart_handler()`.
- Variables that are shared between interrupt and main code, must be marked `volatile` (can change any time, outside direct code execution).

```
#define NBUF 64
static volatile int bufcnt = 0;
static int bufin = 0;
static int bufout = 0;
static volatile char txbuf[NBUF];
static volatile int txidle;
```

# Better Solution: Interrupts

```
void uart_handler(void) {
  if (UART_TXDRDY) {
    UART_TXDRDY = 0;
    if (bufcnt == 0) {
      txidle = 1;
    } else {
      UART_TXD = txbuf[bufout];
      bufcnt--;
      bufout = (bufout+1) % NBUF;
    }
  }
}
```

# Better Solution: Interrupts

```
void serial_putc(char ch) {
  while (bufcnt == NBUF) pause();//volatile ensures fresh read
  intr_disable();
  if (txidle) {
    UART_TXD = ch;
    txidle = 0;
  } else {
    txbuf[bufin] = ch; bufcnt++;
    bufin = (bufin+1) % NBUF;
  }
  intr_enable();
}
```

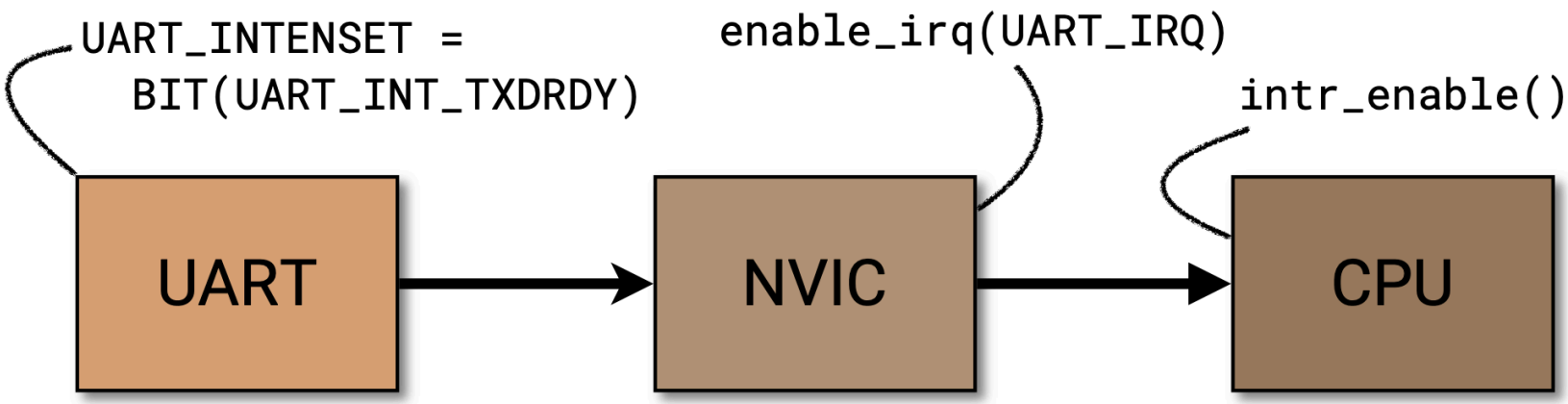
# Why Disable Interrupts?

- What happens if interrupt occurs before `bufcnt++;`?
- What happens if interrupt occurs

```
ldr r0, =bufcnt
ldr r1, [r0]
                        @ <== here?
add r1, r1, #1
str r1, [r0]
```

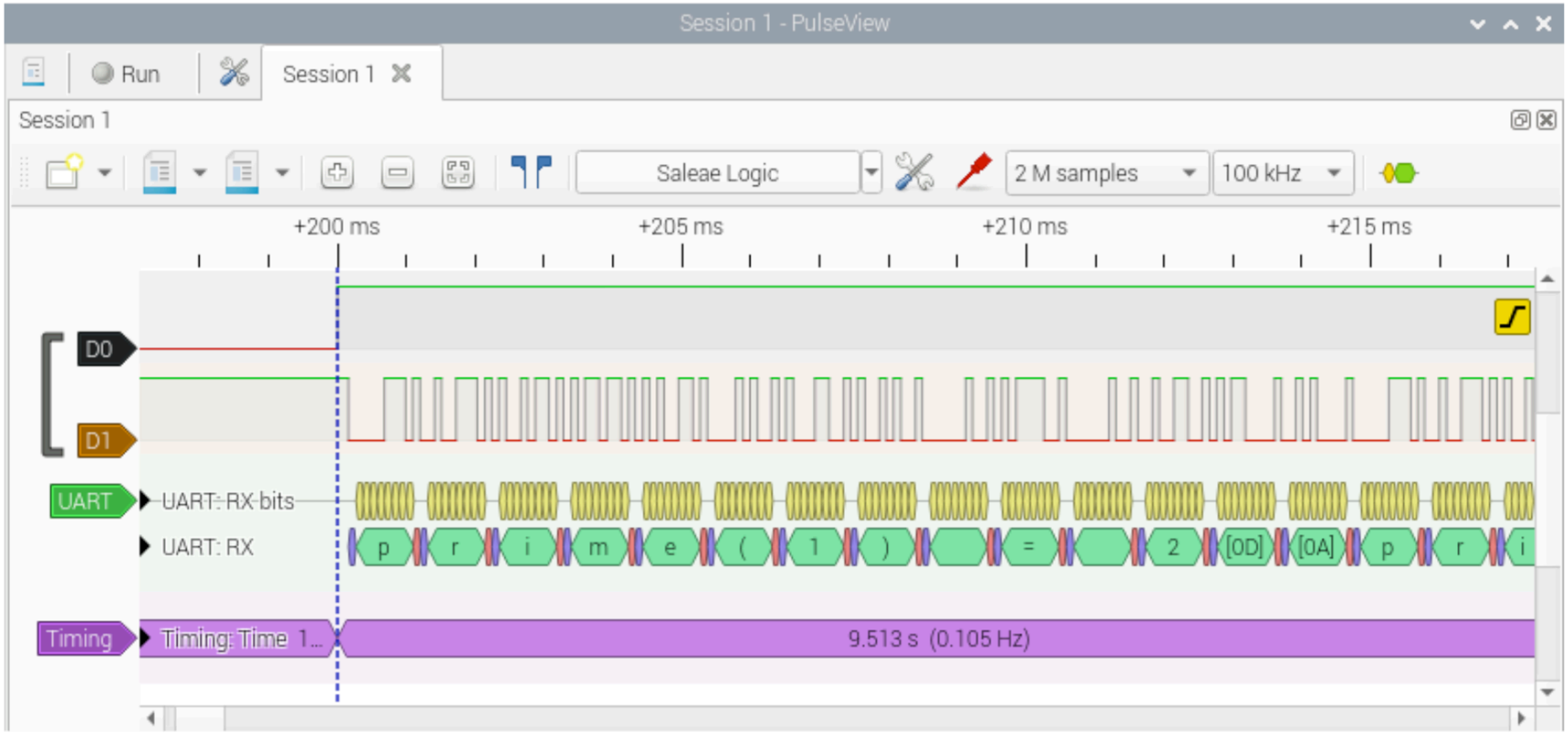
# Setting Up Interrupts

```
void serial_init(void) {
  ...
  UART_INTENSET = BIT(UART_INT_TXDRDY);
  enable_irq(UART_IRQ);
  txidle = 1;
}
```



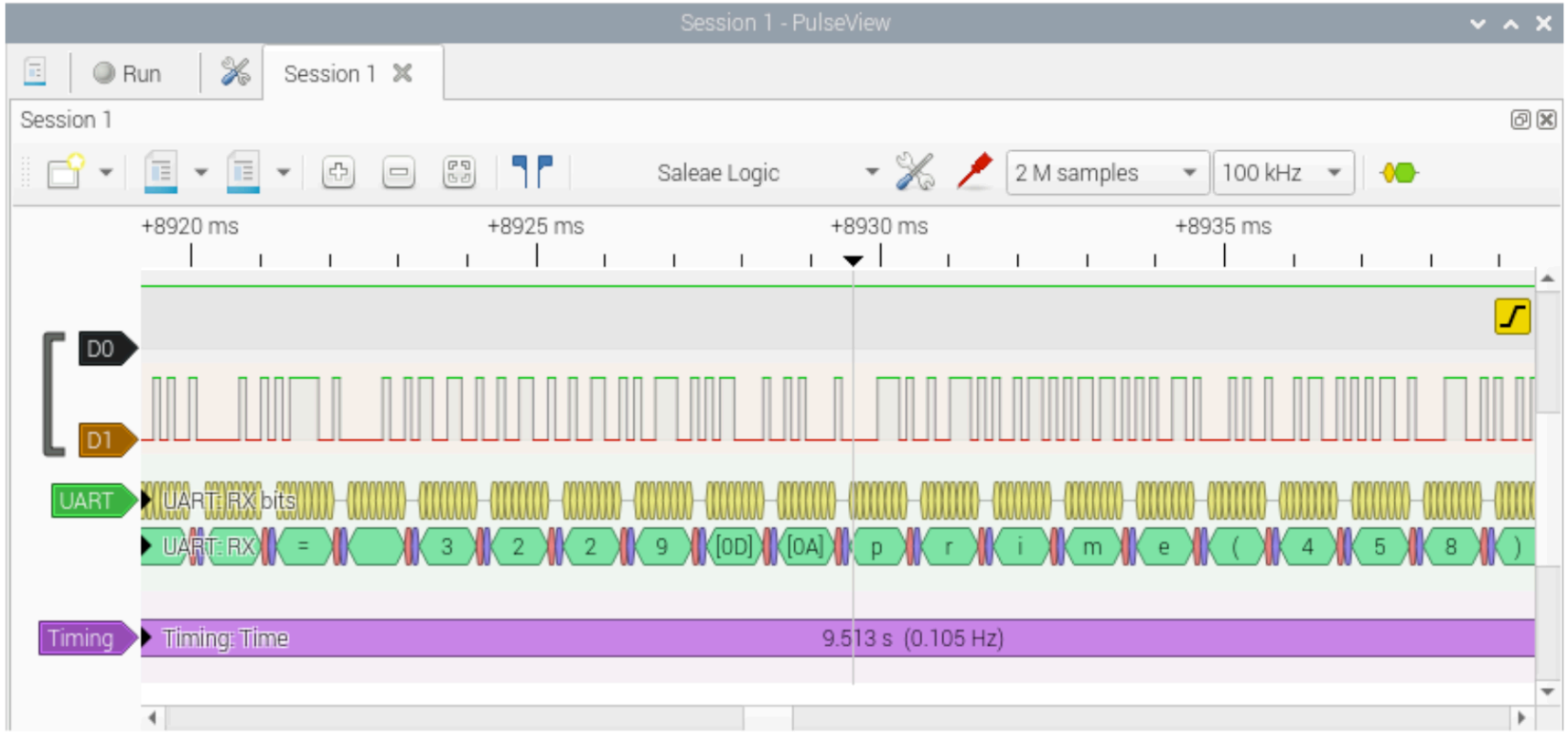
# Timings: Early on in Primes Program

No (noticeable) delays between sending characters.



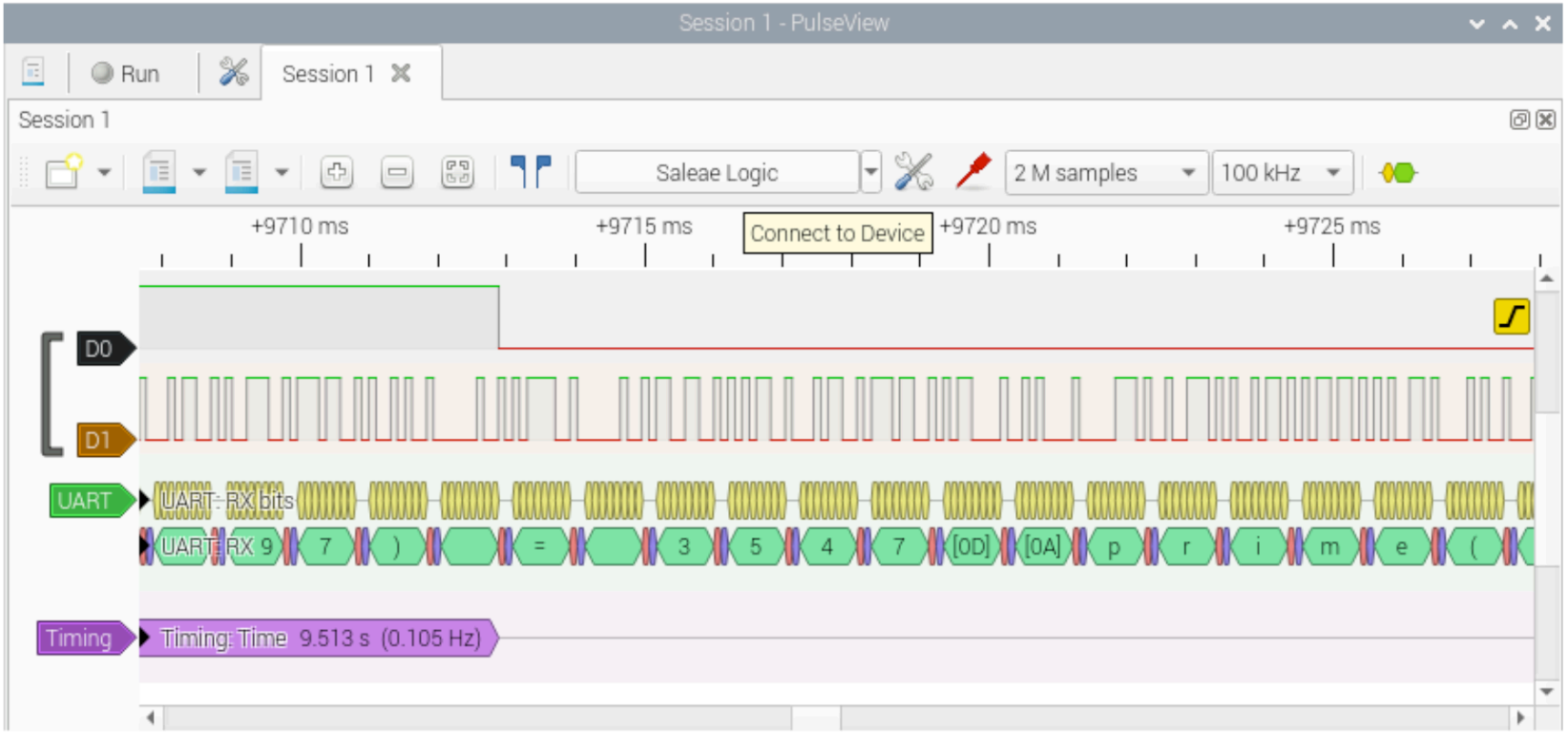
# Timings: Later on in Primes Program

No (noticeable) delays between sending characters.



# Timings: End of Primes Program

Sending characters continues after primes calculation is done!



**Crucial detail omitted!**

## Summary

- How to implement a queue with a circular buffer.
- Why we need interrupts.
- What hardware provides for interrupts.
- How to use interrupts in software.